

Report zur Präsentation – Surprise Adequacy

Malte Grube
Universität Rostock
Rostock, Deutschland
malte.grube@uni-rostock.de

1 Sinn von Surprise Adequacy

In der Softwareentwicklung ist es üblich, strukturierte Metriken wie Branch- oder Statement-Coverage zu verwenden, um die Qualität von Testfällen zu bewerten. In neuronalen Netzen (NNs) gibt es jedoch keine solchen Kontrollflussstrukturen. Somit sind diese Metriken nicht direkt anwendbar [KFY23].

Als frühere Metrik zählt die Neuron Coverage (NC) [Pei+17]. Sie erfasst die Anzahl an Neuronen eines NN, die für die Testdaten einen bestimmten Schwellenwert überschreiten. Diese Metrik ist jedoch nicht fein genug, da die kontinuierlichen Aktivierungen von Neuronen hierbei nicht berücksichtigt werden [KFY19; KFY23].

Surprise Adequacy (SA) [KFY19; KFY23] misst stattdessen, wie "überraschend" eine neue Eingabe für ein neuronales Netzwerk im Vergleich zu den *Trainingsdaten* ist. Dabei werden Activation Traces (ATs) für eine Schicht des Netzwerks betrachtet, für jedes einzelne Input-Bild. Dieser Trace ist ein Vektor, der die Aktivierungen der Neuronen für den Input darstellt. Die Autoren [KFY19; KFY23] empfehlen zudem, dass ein gutes Test-Set neben trainings-ähnlichen Daten auch stark abweichende oder gar adversariale Daten (e.g. manipulierte Bilder via FGSM) enthalten sollte. Mittels SA wurde schließlich geprüft, wie vielfältig eine Test-Suite ist, um die Abdeckung des möglichen Input-Spektrums zu ermitteln.

2 Funktionsweise von SA

SA wird für jeden einzelnen Test-Input basierend auf dem individuellen AT berechnet. Die Berechnung erfolgt der Einfachheit halber für eine ausgewählte Schicht, da die Aktivierungen einer Schicht einen Vektor bilden, der als AT bezeichnet wird.

Bezüglich SA gibt es verschiedene Varianten, welche sich in der Berechnung unterscheiden. Im ersten Paper von [KFY19] werden Likelihood-based Surprise Adequacy (LSA) und Distance-based Surprise Adequacy (DSA) vorgestellt. Im zweiten Paper [KFY23] wird zusätzlich die Mahalanobis Distance-based Surprise Adequacy (MDSA) vorgestellt, welche deutlich effizienter zu berechnen ist.

2.1 LSA

LSA basiert auf der klassenweisen Schätzung der Wahrscheinlichkeitsdichte von ATs mittels Kernel Density Estimation (KDE). Die Dichte wird initial für die ATs der Trainingsdaten geschätzt. Für einen neuen Test-Input wird der AT extrahiert und in der Dichtefunktion ausgewertet. Folglich wird die Überraschung basierend auf der Dichte berechnet:

$$LSA(x) = -\log(\hat{f}(x)).$$

Das bedeutet, je unwahrscheinlicher der Test-AT laut den Trainingsdaten ist (je kleiner die Dichte), desto größer ist das Ergebnis des negativen Logarithmus. Somit sind diese Inputs überraschender, da sie ein ungewöhnliches Aktivierungsmuster aufweisen, welches bei Trainingsdaten selten oder gar nicht vorgekommen ist [KFY23].

2.2 DSA

Um für DSA die klassenspezifische Überraschung des Test-Inputs zu berechnen, werden zwei Werte benötigt: Zuerst wird für das gegebene Testbild die euklidische Distanz im AT-Raum zum nächsten Trainings-AT (dist_a) der gleichen Klasse berechnet. Dann wird die Distanz vom selben Trainings-AT zum nächsten Trainings-AT einer fremden Klasse (dist_b) berechnet. DSA ist folglich das Verhältnis dieser beiden Distanzen:

$$DSA(x) = \frac{\text{dist}_a}{\text{dist}_b}.$$

Je nachdem, wie nah der Test-AT an der Entscheidungsgrenze liegt, desto höher ist grundlegend die DSA [KFY23].

2.3 MDSA

Zur Berechnung von MDSA wird einmalig der Mittelwertsvektor μ und die Kovarianzmatrix C aller Trainings-ATs berechnet. Für einen neuen Test-AT werden diese Elemente genutzt, um die Mahalanobis-Distanz zu berechnen:

$$MDSA(x) = \sqrt{(x - \mu)^T C (x - \mu)}.$$

Liegt der Test-AT nah am Zentrum (Mittelwert) der Trainingsdaten, ist die MDSA niedrig, was als bekanntes Muster interpretiert wird. Liegt der Test-AT weit entfernt, ist die MDSA hoch, was auf ein unbekanntes oder manipuliertes Muster hindeutet.

MDSA wurde als neue Metrik nach LSA und DSA vorgestellt, und ist deutlich schneller als die beiden Metriken, weil die ständigen Vergleiche zwischen neuem Input und den Trainingsdaten entfallen. Dadurch können industrielle Datensätze wie ImageNet sowie große Modelle mittels SA bewertet werden. Bereits das Sampling von 10% der Trainingsdaten reicht bei ImageNet aus, um korrekte Rankings der Test-Inputs zu erhalten, was mit dem größten Speed-Up für MDSA von >9.0 einhergeht [KFY23].

2.4 Surprise Coverage

Im Gegensatz zu NC – Zählung der Neuronen über den Schwellenwert für Test-Inputs – bewertet die Surprise Coverage (SC) die Diversität einer gesamten Test-Suite unter Nutzung der SA-Werte. Nach der Berechnung der SA-Werte aller Test-Inputs wird der kontinuierliche Wertebereich in n Buckets bis zu einer Obergrenze U unterteilt. Somit wird der Wertebereich auf $(0, U]$ beschränkt.

Die SC wird wie folgt ermittelt: Es wird gezählt, in wie viele dieser Buckets mindestens ein Test-Input der Suite fällt, basierend auf dem SA-Wert der Inputs. Anschließend wird diese Anzahl durch die Gesamtzahl der Buckets geteilt, wodurch ein Wert zwischen 0 und 100% herauskommt. Ein hoher SC-Wert bedeutet, dass die Test-Suite eine hohe Diversität aufweist, aufgrund der hohen Anzahl an Buckets, die durch die Test-Inputs abgedeckt werden.

3 Hands-on-Beispiel – DSA mit MNIST

In der Präsentation wurde ein zweidimensionales Hands-on-Beispiel für DSA vorgestellt. Angenommen, es wurden klassenweise für eine bestimmte Modell-Schicht alle ATs für die Trainingsdaten berechnet. Ziel ist es, mit den ATs folglich die DSA-Werte für (1) ein Testbild und für (2) ein FGSM-Bild der Ziffer "5" zu berechnen. Nah zu diesen beiden Bildern liegen die ATs (3) einer "5" und (4) einer "6" aus den Trainingsdaten. Deren ATs werden berechnet und mit den Trainings-ATs verglichen, um die DSA-Werte zu ermitteln. Die Beispielvektoren sind wie folgt gegeben:

- (1) Der Test-AT für "5": (3, 2)
- (2) Der FGSM-AT für "5": (10, 8)
- (3) Nächster Trainings-AT für "5": (2, 2)
- (4) Nächster Trainings-AT für fremde Klasse "6": (6, 2)

Mit diesen ATs können die DSA-Werte für beide Eingabe-Bilder mittels Euklidischer Distanz berechnet werden. Zuerst wird die Distanz vom neuen Test-AT der Zielklasse "5" zum nächsten Trainings-AT dieser Zielklasse berechnet (dist_a). Dann wird die Distanz vom selben Trainings-AT der Zielklasse zum nächsten Trainings-AT der fremden Klasse "6" berechnet (dist_b). DSA ergibt sich aus dem Verhältnis dieser beiden Distanzen:

$$\text{Testbild: } \text{dist}_a = \sqrt{(3-2)^2 + (2-2)^2} = 1$$

$$\text{dist}_b = \sqrt{(6-2)^2 + (2-2)^2} = 4 \quad \text{DSA} = \frac{1}{4} = 0.25$$

$$\text{FGSM-Bild: } \text{dist}_a = \sqrt{(10-2)^2 + (8-2)^2} = 10$$

$$\text{dist}_b = \sqrt{(6-2)^2 + (2-2)^2} = 4 \quad \text{DSA} = \frac{10}{4} = 2.5$$

Das Testbild hat durch die Nähe zum Trainings-AT der Klasse "5" eine niedrige DSA von 0.25. Der AT ist somit ähnlich zu denen der Trainingsdaten, was eine geringe Überraschung folgert. Das FGSM-Bild hingegen hat eine 10-Mal so hohe DSA von 2.5, da es weiter von den Trainings-ATs der Zielklasse "5" entfernt ist, und näher an den ATs der fremden Klasse "6" liegt. Der AT des FGSM-Bildes ist somit weniger gewöhnlich im Vergleich zu den Trainingsdaten.

Dies kann zu Fehlklassifikationen führen, da ungewöhnliche ATs für FGSM-basierte Bilder vorliegen. Als Beispiel dient Abbildung 1.

4 Repository-Beispiel – LSA mit MNIST

Die offizielle Implementierung von SA¹ bietet die Möglichkeit, beispielsweise LSA auf dem MNIST-Datensatz anzuwenden. Das Repository bietet unter anderem eine Trainings-Pipeline für ein einfaches Convolutional Neural Network (CNN) sowie die Berechnung von LSA für Test- oder FGSM-Bilder. Es wurde für den Report mit Python 3.5.10, pip 9.0.1 und Linux 6.19.11-arch1-1 getestet.

Bereits 3 Epochen (statt 50 wie im Repository) an Training mit dem gegebenen 5-Schichten-CNN haben für MNIST ausgereicht, eine Validation Accuracy von 98.78% zu erreichen. Danach wurden die ATs für Trainings-, Test- und FGSM-Bilder berechnet, um anschließend die LSA-Werte für Test- und FGSM-Bilder – hinsichtlich des Training-Sets – zu ermitteln. Die Geschwindigkeit der LSA-Berechnung wurde verbessert, indem im Code durch die Autoren irrelevante Neuronen, basierend auf einem Schwellenwert [KFY19],

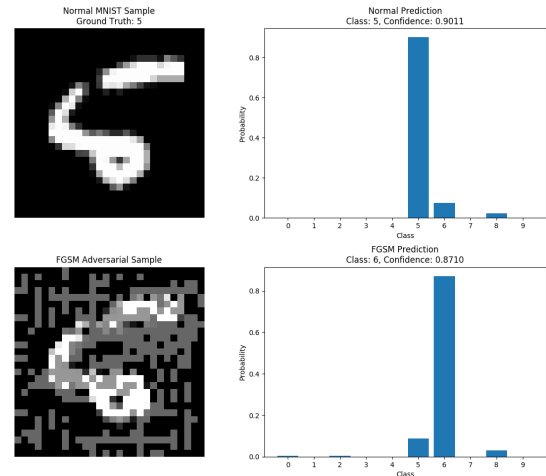


Abbildung 1: Vorhersage einer "5". Das Test-Bild wurde mit hoher Wahrscheinlichkeit korrekt klassifiziert, während das FGSM-Bild stattdessen als "6" klassifiziert wurde.

herausgefiltert wurden. Im zweiten Paper [KFY23] wird empfohlen, dies auch auf DSA und MDSA anzuwenden, auch wenn es nicht zwingend erforderlich ist.

Ein Vergleich der LSA-Werte für Test- und FGSM-Bilder zeigt, dass die FGSM-Bilder klassenweise oftmals breiter und nach rechts verschoben verteilt sind, was eine Tendenz zu höheren LSA-Werten aufweist (siehe Abbildung A.1). Der Mean für Test beträgt -52.31, während FGSM einen Mean von 5.53 aufweist. In Abbildung A.2 wirken die akkumulierten Ergebnisse für LSA nicht bahnbrechend, zumal ausschließlich LSA getestet wurde. DSA oder gar MDSA wurden hingegen nicht getestet, da in diesem Repository LSA mit MNIST als primäres Beispiel schnell anwendbar ist.

Hinsichtlich der SC weist die Vermischung beider Datensätze einen höheren Wert auf. Die Test-Suite zeigt lokale 27.7% SC, während die Kombination aus Test- und FGSM-Bildern 31.7% SC aufweist. In [KFY23] wurde die SC für LSA durch weitere Adversarial-Datensätze auf bis zu 71% erhöht, was die Diversität der Test-Suite deutlich steigert. Aufgrund der 3 Epochen an Training fallen die SC-Werte von den Ergebnissen etwas geringer als in [KFY19] aus. Mehr Informationen zum Repository gibt es im Anhang B.

5 Schlusswort

Die Ergebnisse zu SA verdeutlichen, dass eine systematische Auswahl an Bildern über das LSA-Spektrum hinweg (von ähnlich zu Trainingsdaten bis hin zu stark abweichend) sinnvoll sein kann, um das Modell mittels Retraining zu verbessern [KFY19]. Das Nachfolgepaper [KFY23] zeigt jedoch, dass der Retraining-Effekt in den umfassenderen Experimenten zu SA nicht allzu stark ist. SA reicht allein nicht aus, um ein gutes Retraining zu gewährleisten.

Es wird dennoch eine ausreichend schnelle Pipeline geboten, um beispielsweise LSA auf gängigen Datensätzen wie MNIST anzuwenden. Zuvor haben die Autoren das MNIST-Testset mit FGSM manipuliert, und dieses zur Verfügung gestellt, siehe Abbildung 1.

Grundlegend kann SA "überraschende" Inputs erkennen, jedoch nicht erklären, warum ein Modell die Entscheidung für jene Inputs trifft, im Vergleich zu Explainable AI (XAI) Methoden.

¹www.github.com/coinse/sadl, letzter Zugriff 04/05/2026

A LSA Histogramme

Die folgenden LSA-Histogramme wurden zur besseren visuellen Differenzierung im positiven Bereich bis $x \leq 200$ dargestellt.

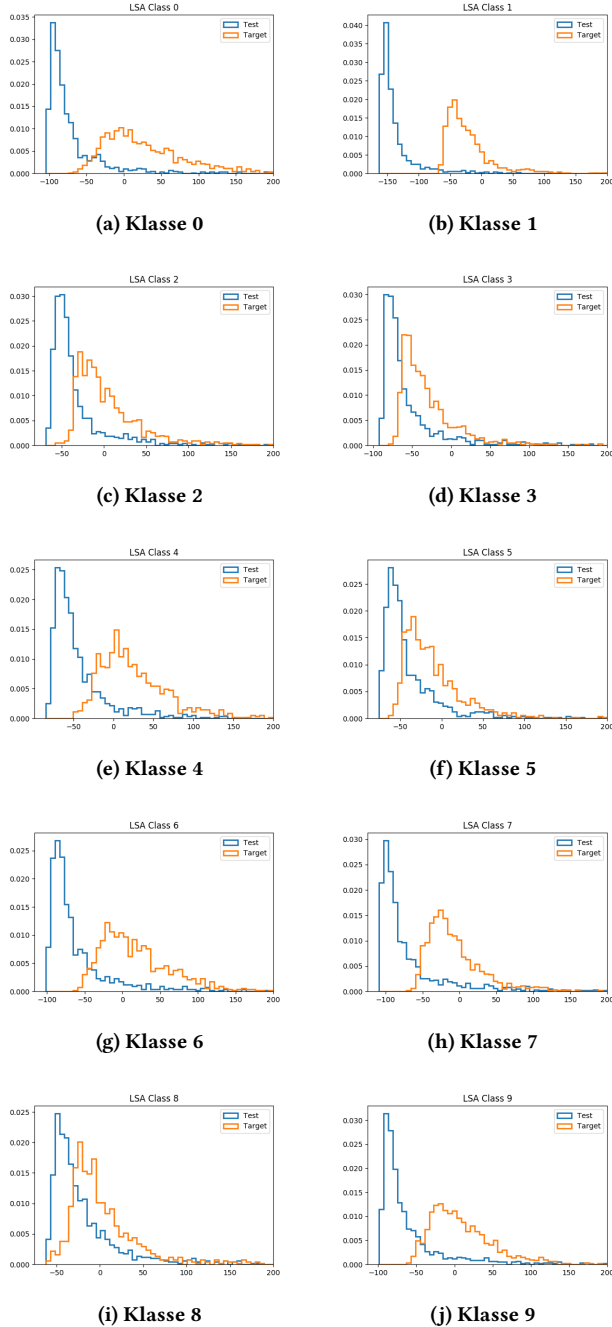


Abbildung A.1: Übersicht der LSA-Histogramme pro MNIST-Klasse (0-9) für das Testset. Die LSA-Verteilungen sind zum Vergleich pro Klasse sowohl für originale als auch FGSM-manipulierte Bilder dargestellt.

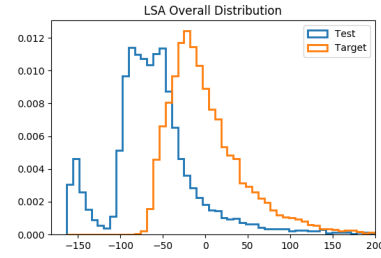


Abbildung A.2: Akkumulierte LSA-Verteilungen für das MNIST-Testset zwischen Original- und FGSM-Bildern.

B Zum Source Code

Der Code zum Report befindet sich in einem GitHub-Fork² zur offiziellen Implementierung. Es wurden Anpassungen vorgenommen, um beispielsweise das **fehlende** Tensorboard-Logging für das Training, für die LSA- und SC-Berechnungen zu ermöglichen, siehe Abbildungen A.1 und A.2. Zudem wurde ein Python-Modul (`show_examples.py`) hinzugefügt, um die Vorhersage aus Abbildung 1 zu präsentieren. Änderungen zum Originalcode sind durch Beginn- und End-Kommentare markiert (sowie durch Git Diffs ersichtlich), vorwiegend in `run.py` zu finden.

Wie in [KFY19; KFY23] angegeben, kann sich die SA-Berechnung auf anderen DL-Architekturen abweichend verhalten, wie im jeweiligen "Threats to Validity" Kapitel zu sehen. Daraus ergibt sich die Frage, wieso viele Aspekte nicht in der Implementierung berücksichtigt wurden, da diverse Paper-Details darin nicht abgedeckt wurden. Jegliches Seed-Management für eine *geeignete Reproduzierbarkeit* der Ergebnisse ist zudem nicht implementiert worden, weshalb die Ergebnisse des Reports nicht mit den Ergebnissen im Paper übereinstimmen können. Dies gilt sowohl für das durchgeführte 3-Epochen-Training (ausreichende Validation Accuracy), als auch für ein womöglich verlangtes 50-Epochen-Training. Die Anzahl der Epochen wurde in den Papern nicht dargelegt, aber die standardmäßigen 50 Epochen sollten korrekt sein. Jegliche Ergebnisse können trotzdem anders ausfallen, da die Modellparameter entscheidend für die SA-Werte sind. Des Weiteren ist das Ermitteln von SC-Werten nur teilweise implementiert worden: entweder wird die Coverage für die gesamte Test-Suite oder für die gesamte FGSM-basierte Suite bestimmt. Keinerlei Kombination beider Suiten (wie in beiden Veröffentlichungen) ist möglich gewesen. Dies musste eigenständig hinzugefügt werden. Die Werte bezüglich des 3-Epochen-Trainings sind etwas geringer als in [KFY19], aber zeigen ähnliche Tendenzen.

Die Autoren haben zudem, im Gegensatz zum geringfügig abdeckenden Material, in ihren Papern weitere Adversarial-Datensätze (wie BIM, JSMA, ...) vorgestellt, um schrittweise die SC-Werte zu erhöhen. Dies geschah durch iteratives Hinzufügen von 1.000 manipulierten Bildern pro Adversarial-Datensatz ($\text{Test} + \text{FGSM} \rightarrow \dots + \text{BIM-A, etc.}$) [KFY19]. Im Repository werden lediglich entweder alle 10.000 FGSM-Bilder oder Test-Bilder auf einmal betrachtet, ohne eine Vermischung beider Sets. Zusammenfassend, viele Paper-Details wurden nicht im Repository abgedeckt, und wurden teilweise eigenständig hinzugefügt, um ausreichend demonstrieren zu können.

²<https://github.com/mgrube753/sadl-se4ai>

Literatur

- [KFY19] Jinhan Kim, Robert Feldt und Shin Yoo. “Guiding Deep Learning System Testing Using Surprise Adequacy”. In: *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. 2019, S. 1039–1049. DOI: 10.1109/ICSE.2019.00108.
- [KFY23] Jinhan Kim, Robert Feldt und Shin Yoo. “Evaluating Surprise Adequacy for Deep Learning System Testing”. In: *ACM Trans. Softw. Eng. Methodol.* 32.2 (März 2023). ISSN: 1049-331X. DOI: 10.1145/3546947. URL: <https://doi.org/10.1145/3546947>.
- [Pei+17] Kexin Pei u. a. “DeepXplore: Automated Whitebox Testing of Deep Learning Systems”. In: *Proceedings of the 26th Symposium on Operating Systems Principles. SOSP ’17*. Shanghai, China: Association for Computing Machinery, 2017, S. 1–18. ISBN: 9781450350853. DOI: 10.1145/3132747.3132785. URL: <https://doi.org/10.1145/3132747.3132785>.